Fase 4 — Justificación de decisiones de diseño

Este documento justifica las 11 decisiones de diseño más importantes del modelo UML v1. Para cada una indicamos las alternativas que consideramos, por qué las descartamos y qué principio respalda la elección. La decisión central es aislar la API de clima detrás de la interfaz `ProveedorClima`, que se inyecta en `AsignadorEntregas`: así el proveedor se puede cambiar sin tocar la lógica de asignación (inversión de dependencias).

1. Resumen de decisiones

#	Decisión	Alternativas consideradas	Elección	Principio
D1	Tipos de robot	(a) Una clase `Robot` con atributo `tipo` y `if`. (b) Jerarquía `Robot` → `RobotRuedas`, `RobotOrugas`, `Dron`. (c) Composición de módulos de locomoción.	(b) Herencia	Abierto/cerrado, sustitución de Liskov
D2	Batería	(a) Atributo `float` en `Robot`. (b) Clase `Bateria` compuesta en `Robot`.	(b) Composición	Responsabilidad única, encapsulamiento
D3	Tipos de carga	(a) Enum `TipoCarga` con las reglas en el asignador. (b) Jerarquía `Carga` con `recargo()` y `es_compatible_con()`.	(b) Herencia	Abierto/cerrado, alta cohesión
D4	Compatibilidad carga–robot	(a) La carga pregunta la subclase del robot (`isinstance`). (b) El robot expone capacidades y la carga las consulta.	(b) Capacidades	Bajo acoplamiento

#	Decisión	Alternativas consideradas	Elección	Principio
D5	Integración del clima	(a) Llamar a la API dentro del asignador. (b) Clase concreta `ServicioClima`. (c) Interfaz `ProveedorClima` inyectada.	(c) Interfaz + inyección	Inversión de dependencias, abierto/cerrado
D6	API de clima caída	(a) Propagar la excepción. (b) Asumir buen clima. (c) Asumir mal clima y registrarlo.	(c) Fail-safe	Robustez, ocultamiento de información
D7	Dónde vive la asignación	(a) En `Flota`. (b) En cada `Robot`. (c) Clase `AsignadorEntregas`.	(c) Clase dedicada	Responsabilidad única, alta cohesión
D8	Relación Flota–Robot	(a) Composición. (b) Agregación.	(b) Agregación	Modelado fiel del dominio
D9	Relación Bitácora–Evento	(a) Eventos creados afuera. (b) La bitácora crea sus eventos.	(b) Composición	Encapsulamiento, integridad
D10	Umbrales de clima	(a) Constantes dentro del servicio. (b) Objeto `UmbralesClima`.	(b) Objeto de configuración	Separación de configuración, no repetir
D11	Interfaz gráfica	(a) Reglas en los botones. (b) Interfaz delgada que usa el asignador.	(b) Interfaz delgada	Separación en capas

2. Decisiones explicadas

D1. Tipos de robot: herencia

Contexto. La flota tiene robots de ruedas, de orugas y drones. Cada uno cambia la velocidad, las zonas a las que llega, el costo por kilómetro y si necesita consultar el clima.

Alternativas.

- (a) Una sola clase `Robot` con un atributo `tipo` y condicionales.
- (b) Una clase abstracta `Robot` con las subclases `RobotRuedas`, `RobotOrugas` y `Dron`.
- (c) Un `Robot` compuesto por un módulo de locomoción intercambiable.

Por qué descartamos las otras. Con (a), cada regla nueva obliga a editar todos los condicionales y un error en uno afecta a todos los tipos. La opción (c) es más flexible, pero para tres tipos estables agrega piezas que no necesitamos y hace el modelo más difícil de explicar.

Elección y principio. (b). Un dron *es un* robot: la relación es real y el asignador trata a todos como `Robot`. Agregar un tipo nuevo es agregar una subclase sin modificar las existentes (abierto/cerrado), y cualquier subtipo puede usarse donde se espera un `Robot` (Liskov).

Dónde se ve. Tarjetas CRC "Robot" y "RobotRuedas / RobotOrugas / Dron"; en el UML, la herencia hacia `Robot` y los métodos abstractos `zonas_accesibles()`, `costo_por_km()` y `consumo_por_km()`.

D2. Batería: composición

Contexto. La batería tiene reglas: el nivel va de 0 % a 100 % y un robot debe terminar cada entrega con al menos 15 % (supuesto S3).

Alternativas.

- (a) Un atributo `float` dentro de `Robot`.
- (b) Una clase `Bateria` que cada robot crea y contiene.

Por qué descartamos (a). Las validaciones quedarían repartidas entre `Robot` y el asignador, y cualquiera podría dejar el nivel en un valor inválido.

Elección y principio. (b). `Bateria` es responsable de su propio estado (responsabilidad única) y lo protege con métodos (`consumir()`, `recargar()`, `alcanza_para()`). Es composición porque la batería nace y muere con su robot.

Dónde se ve. Tarjeta CRC "Bateria"; en el UML, `Robot ◆— Bateria` con multiplicidad 1–1.

D3. Tipos de carga: herencia

Contexto. Documentos, alimentos, objetos frágiles y materiales peligrosos tienen restricciones y recargos distintos.

Alternativas.

- (a) Un enum `TipoCarga` y las reglas de cada tipo dentro del asignador.
- (b) Una clase abstracta `Carga` con `recargo()` y `es_compatible_con(robot)`, redefinidos por cada subtipo.

Por qué descartamos (a). El asignador terminaría conociendo las reglas de todas las cargas (baja cohesión) y cada carga nueva obligaría a modificarlo.

Elección y principio. (b). Cada carga decide sus propias reglas. Agregar una carga nueva, por ejemplo un medicamento, es crear una subclase sin tocar el asignador ni los robots (abierto/cerrado).

Dónde se ve. Tarjeta CRC "Carga"; en el UML, `Carga` → `Documento`, `Alimento`, `Fragil`, `MaterialPeligroso`.

D4. Compatibilidad entre carga y robot: capacidades

Contexto. Un alimento necesita control de temperatura, un frágil no puede ir en orugas y un material peligroso nunca va en dron.

Alternativas.

- (a) La carga pregunta de qué subclase es el robot (`isinstance(robot, Dron)`).
- (b) El robot expone sus capacidades (`es_aereo()`, `terreno_irregular()`, `_tiene_control_temperatura`) y la carga las consulta.

Por qué descartamos (a). Ata cada carga a cada subclase de robot: un robot nuevo obligaría a revisar todas las cargas.

Elección y principio. (b). Un robot nuevo solo declara sus capacidades y las cargas siguen funcionando (bajo acoplamiento). Por esta decisión agregamos `_tiene_control_temperatura` a `Robot` al pasar de las CRC al UML.

Dónde se ve. En el UML, los métodos de capacidad de `Robot` y `es_compatible_con()` en las cargas; el cambio está registrado en `CHANGELOG_UML.md`.

D5. Integración de la API de clima: interfaz inyectada

Contexto. Antes de asignar un dron hay que consultar el clima en Open-Meteo. El PRD exige que el proveedor se pueda sustituir sin modificar la lógica de asignación.

Alternativas.

- (a) Hacer la petición HTTP directamente dentro de `AsignadorEntregas`.
- (b) Una clase concreta `ServicioClima` que el asignador crea y usa.
- (c) Una interfaz `ProveedorClima` con dos implementaciones (`ServicioClimaOpenMeteo` y `ServicioClimaSimulado`), que el asignador recibe por el constructor.

Por qué descartamos las otras. Con (a), las reglas de negocio se mezclan con HTTP y JSON, y no se pueden probar sin internet. Con (b), la API queda aislada, pero el asignador sigue dependiendo de una clase concreta: cambiar de proveedor obliga a modificarlo.

Elección y principio. (c). El asignador depende de una abstracción y no de Open-Meteo (inversión de dependencias). Un proveedor nuevo es una clase más que implementa la interfaz (abierto/cerrado). Es una asociación y nunca herencia, porque el asignador no *es un* servicio de clima.

Dónde se ve. Tarjetas CRC "AsignadorEntregas" y "ProveedorClima"; en el UML, la asociación «inyectado» entre `AsignadorEntregas` y `ProveedorClima`. La sección 3 muestra la sustitución.

D6. Qué pasa si la API de clima falla: fail-safe

Contexto. Open-Meteo puede no responder, tardar demasiado o devolver datos inválidos.

Alternativas.

- (a) Dejar que la excepción llegue hasta el asignador y la interfaz.
- (b) Asumir que el clima es bueno y dejar volar al dron.
- (c) Devolver un reporte "no disponible", tratarlo como mal clima y registrarlo en la bitácora.

Por qué descartamos las otras. Con (a), un problema externo tumba la aplicación. Con (b), arriesgamos perder un dron o lastimar a alguien.

Elección y principio. (c). El error se maneja dentro de `ServicioClimaOpenMeteo`, que oculta los detalles de la falla (ocultamiento de información). El sistema sigue funcionando: la entrega se hace por tierra o queda pendiente (supuesto S4, timeout de 5 s).

Dónde se ve. En el UML, `ReporteClima._disponible` y `ServicioClimaOpenMeteo._timeout_s`.

D7. Dónde viven las reglas de asignación: clase dedicada

Contexto. Asignar una solicitud implica revisar la flota, consultar el clima, elegir el robot más barato y registrar el resultado.

Alternativas.

- (a) Ponerlo en `Flota`.
- (b) Que cada `Robot` decida si toma la solicitud.
- (c) Una clase `AsignadorEntregas`.

Por qué descartamos las otras. Con (a), la flota tendría que hablar con el clima y la bitácora (baja cohesión). Con (b), un robot tendría que conocer al resto de la flota y al clima.

Elección y principio. (c). El robot sabe si *puede* atender (`puede_atender()`); el asignador *elige* entre los que pueden. Cada clase tiene una sola razón para cambiar (responsabilidad única).

Dónde se ve. Tarjeta CRC "AsignadorEntregas"; en el UML, `AsignadorEntregas` asociado a `Flota`, `ProveedorClima`, `Bitacora` y `SolicitudEntrega`.

D8. Relación entre Flota y Robot: agregación

Alternativas. (a) Composición: la flota crea y destruye sus robots. (b) Agregación: la flota agrupa robots que existen por sí mismos.

Por qué descartamos (a). Un robot puede salir de la flota (venta, préstamo a otra sede) y seguir existiendo. La flota no controla su ciclo de vida.

Elección y principio. (b). El modelo refleja el dominio real.

Dónde se ve. En el UML, `Flota ◇— Robot` con multiplicidad 0..1–0..*.

D9. Relación entre Bitácora y Evento: composición

Alternativas. (a) Los eventos se crean afuera y se agregan a la bitácora. (b) La bitácora crea sus propios eventos con `registrar(tipo, mensaje)`.

Por qué descartamos (a). Cualquier clase podría crear o modificar eventos y alterar el historial.

Elección y principio. (b). Los eventos solo existen dentro de la bitácora y `eventos()` devuelve una copia, así el historial no se puede alterar desde afuera (encapsulamiento, integridad).

Dónde se ve. Tarjetas CRC "Bitacora" y "EventoBitacora"; en el UML, `Bitacora ◆— EventoBitacora`.

D10. Umbrales de clima: objeto de configuración

Alternativas. (a) Constantes dentro de `ServicioClimaOpenMeteo`. (b) Un objeto `UmbralesClima` que se entrega al proveedor.

Por qué descartamos (a). Cambiar un umbral obligaría a editar código, y el simulador tendría que repetir los mismos valores.

Elección y principio. (b). El proveedor real y el simulado usan los mismos límites (30 km/h de viento, 40 km/h de ráfagas, 50 % de lluvia) sin repetirlos. En las CRC estaban junto con el reporte de clima; en el UML los separamos porque tienen ciclos de vida distintos.

Dónde se ve. En el UML, `UmbralesClima` asociado a `ProveedorClima`; el cambio está en `CHANGELOG_UML.md`.

D11. Interfaz gráfica: interfaz delgada

Alternativas. (a) Poner las reglas en las funciones de los botones. (b) Una interfaz que solo llama a `AsignadorEntregas` y muestra `Flota` y `Bitacora`.

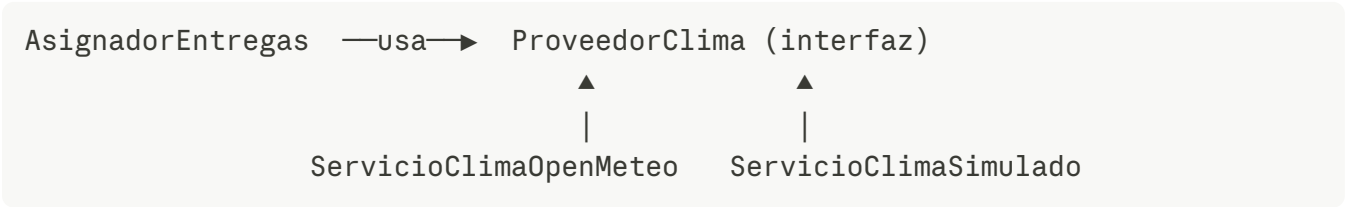
Por qué descartamos (a). Las reglas no se podrían probar sin abrir ventanas y quedarían duplicadas entre la interfaz y el dominio.

Elección y principio. (b). El dominio no sabe que existe una interfaz; las pruebas cubren las reglas y la interfaz se puede reemplazar (separación en capas).

Dónde se ve. En las CRC, "Interfaz gráfica" aparece en lo que queda fuera del modelo de dominio.

3. Sustitución del proveedor de clima

El PRD pide que el proveedor de clima se pueda cambiar (otra API o un simulador para pruebas) sin modificar la lógica de asignación. Con la decisión D5 la dependencia queda así:



El asignador solo conoce `ProveedorClima`. El proveedor concreto se decide afuera y se le entrega:

```
umbrales = UmbralesClima(viento_max_kmh=30, rafagas_max_kmh=40,
lluvia_max_pct=50)

# Uso normal: clima real
asignador = AsignadorEntregas(flota, ServicioClimaOpenMeteo(umbrales),
bitacora)

# Prueba o defensa: clima simulado con viento fuerte
asignador.cambiar_proveedor_clima(ServicioClimaSimulado(umbrales, viento=45))
```

Qué no cambia: el código de `AsignadorEntregas`. Sigue llamando a `condiciones_favorables(ubicacion)` sin saber si detrás está Open-Meteo o el simulador.

Para otra API de clima: se crea una clase nueva que implementa `ProveedorClima` y se le entrega al asignador. Ninguna clase existente se modifica.

4. Principios aplicados

Principio	Qué significa en este proyecto	Decisiones
Responsabilidad única	Cada clase tiene una sola razón para cambiar.	D2, D7
Abierto/cerrado	Agregar un robot, una carga o un proveedor no modifica clases existentes.	D1, D3, D5
Sustitución de Liskov	Cualquier subtipo de `Robot` funciona donde se espera un `Robot`.	D1
Inversión de dependencias	El asignador depende de `ProveedorClima`, no de Open-Meteo.	D5
Bajo acoplamiento y alta cohesión	Las cargas dependen de capacidades, no de subclases; cada regla vive con sus datos.	D3, D4, D7
Encapsulamiento	Batería y bitácora protegen su propio estado.	D2, D9

5. Límites conocidos del diseño

- Si aparecieran robots híbridos (por ejemplo, terrestres que también vuelan), la herencia de D1 se quedaría corta. La evolución natural sería pasar a composición de capacidades, que hoy descartamos por simplicidad.
- La distancia se calcula en línea recta con un factor de rodeo de 1,3 (supuesto S5). Si se necesitaran rutas reales, iría detrás de su propia interfaz, igual que el clima.
- Elegimos el robot compatible más barato. Si la universidad definiera prioridades entre solicitudes (pregunta 2 al cliente), habría que revisar esa regla.